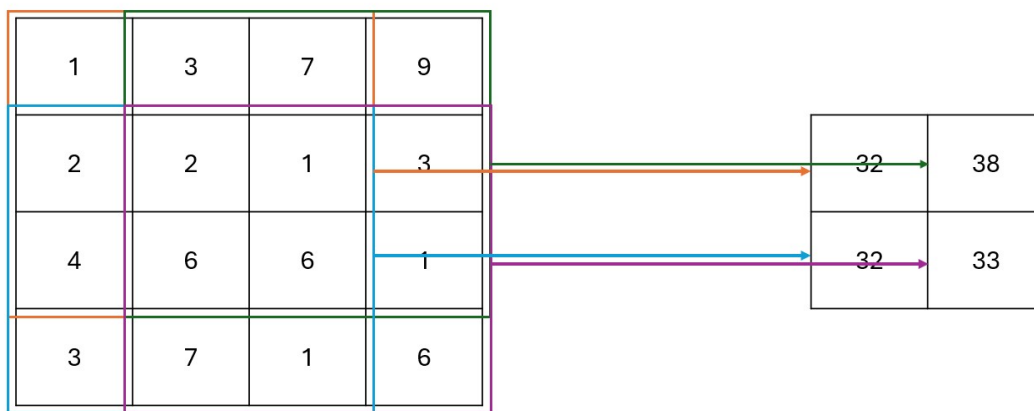
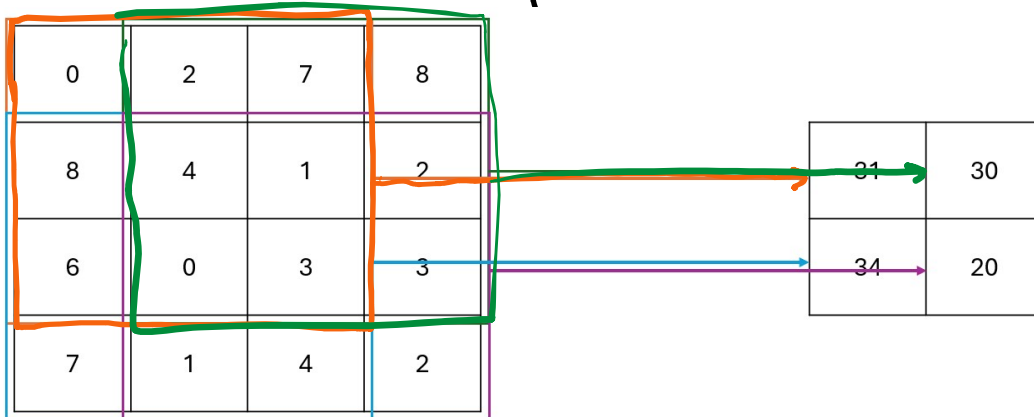


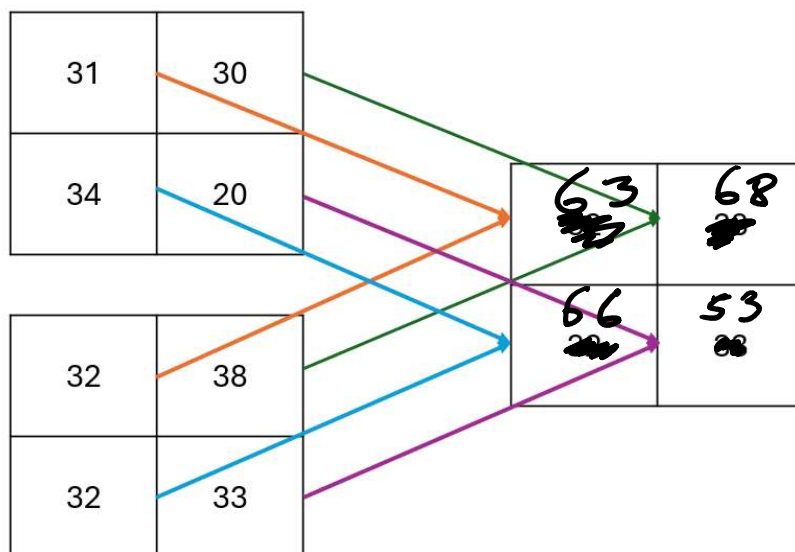
Backpropagation on Convolution

February 25, 2026 8:32 AM

Example

$$w(s, t, c) = \begin{cases} 1 & \text{if } -1 \leq s, t \leq 1 \\ 0 & \text{otherwise} \end{cases}$$





Backprop on Convolution

$$w_i \leftarrow w_i - \alpha \frac{\partial L}{\partial w_i}$$

w_i - weights in the kernel.

Output of convolution is

$$O_t = (x * w)(t) = \sum_{a=-\infty}^{\infty} x(t+a)w(a)$$

For a fixed width kernel of width k

$$O_t = w_{-\frac{k-1}{2}} x_{t-\frac{k-1}{2}} + \dots + w_0 x_t + \dots +$$

$$w_{\frac{k-1}{2}} x_{t + \frac{k-1}{2}}$$

if $k=3$

$$o_t = w_{-1} x_{t-1} + w_0 x_t + w_1 x_{t+1}$$

$$\frac{\partial L}{\partial w_i} = \sum_t \left(\frac{\partial L}{\partial o_t} \right) \left(\frac{\partial o_t}{\partial w_i} \right)$$

given by subsequent layer

$$\frac{\partial o_t}{\partial w_i} = x_{t+i}$$

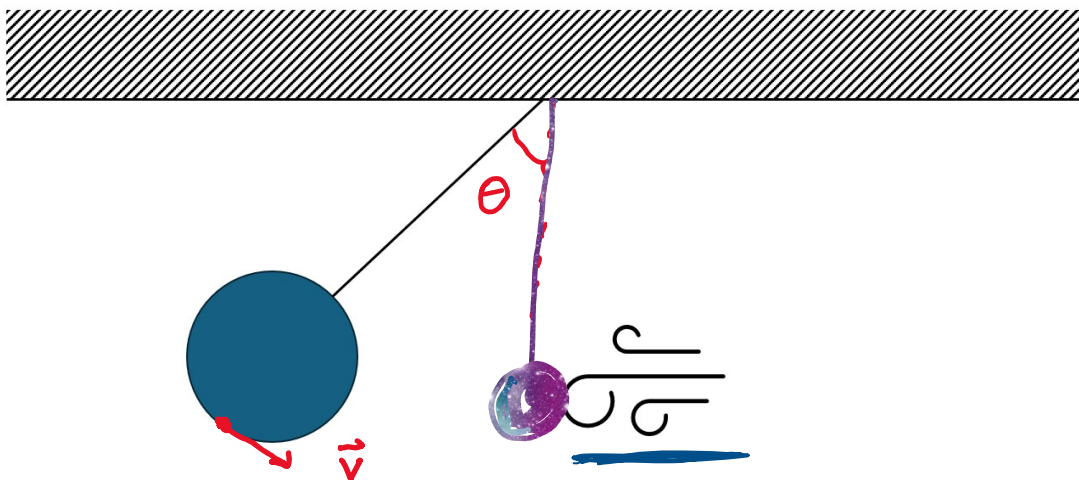
To facilitate backprop

$$\frac{\partial L}{\partial x_i} = \sum_t \left(\frac{\partial L}{\partial o_t} \right) \left(\frac{\partial o_t}{\partial x_i} \right)$$

given by subsequent layer

$$\frac{\partial o_t}{\partial x_i} = w_{i-t}$$

Recurrent Neural Networks



Model how system evolves over time.

1. State space (the set of possible states)
2. Time evolution function (how system changes over time).

$$s_t = f(s_{t-1}; \theta)$$

f, θ - constants

The state is driven by external signal

$$s_t = f(s_{t-1}, x_t; \theta)$$

RNNs are a neural network architecture designed to process sequential data.

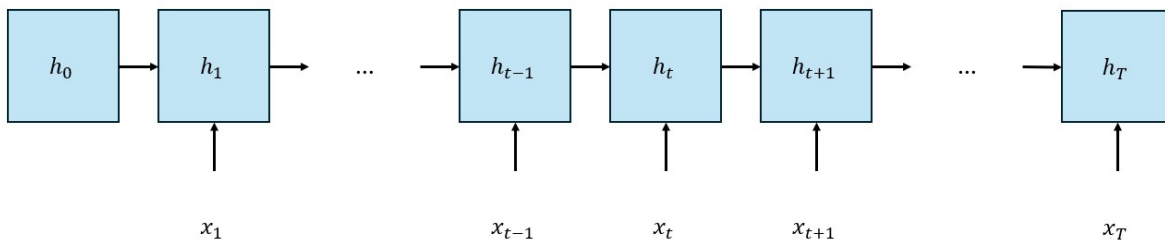
$$h_t = f(h_{t-1}, x_t; \Theta)$$

The state h_t summarizes the system

- initial state h_0
- prior values of external signal $x_1, \dots, x_t$

Goal: Have hidden state h_t capture info about external signal relevant to some task.

Tune parameters Θ to achieve this.

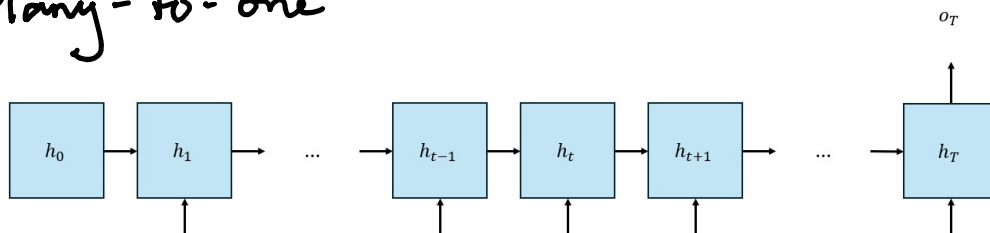


Parameter Sharing

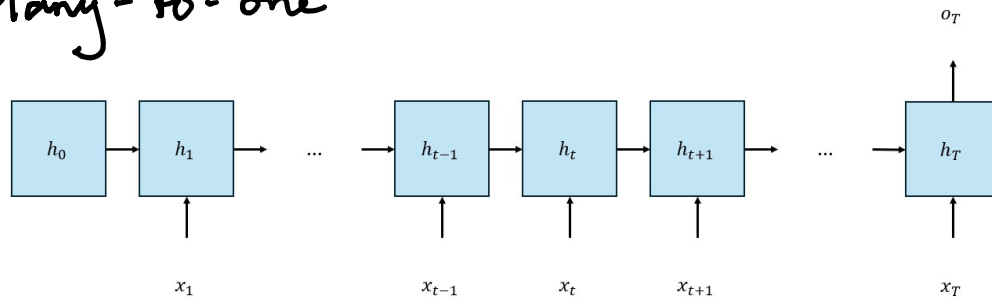
1. Fewer parameters to learn.

2. Characteristics of sequence learned independent of time.

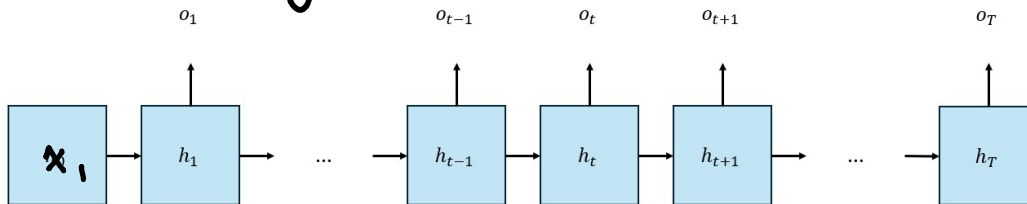
Many-to-one



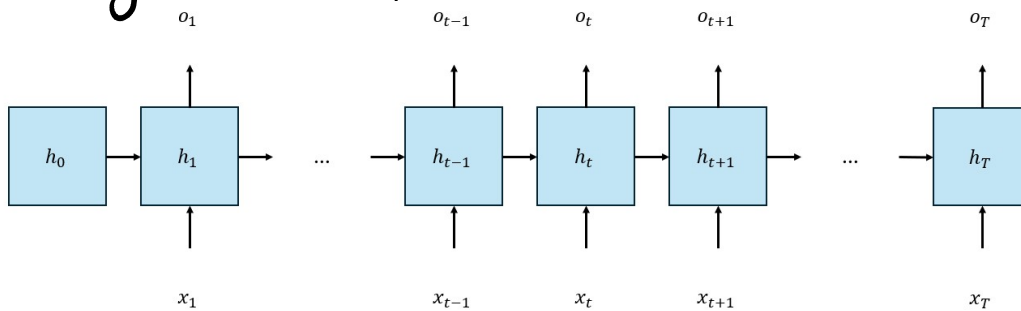
Many-to-one



One-to-many:



Many-to-many:



Vanilla Recurrent Unit

$$f(h_{t-1}, x_t; \theta) = ?$$

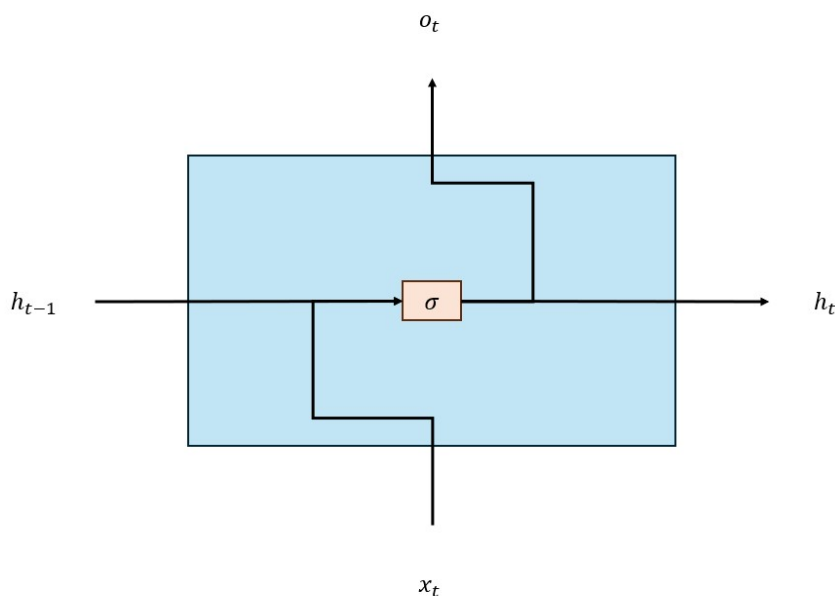
$$a_t = b + W h_{t-1} + U x_t$$

$$h_t = \tanh(a_t)$$

$$o_t = c + V h_t$$

b, c - vector parameters

b, c - vector parameters
 W, u, v - matrix parameters



This does not handle long-term dependencies well.

$$h_t \approx W h_{t-1} \approx W^2 h_{t-2} \approx \dots \approx W^t h_0$$

If compute eigen decomposition of
 $W = Q D Q^T$

$$h_t \approx (Q D Q^T)^t h_0$$

$$h_t \approx Q D^t Q^T h_0$$

- If Q is the matrix of eigenvectors of

Components of h_0 aligned with eigenvectors of W with $\lambda > 1$ will exponentially increase

Components of h_0 aligned with eigenvectors of W with $\lambda < 1$ will exponentially decrease

Vanilla recurrent unit cannot model long-term dependencies.